

CS 4390 Computer Networks

P2P File Sharing Course Project Final Report

Submitted by: Diep, Ethan; Giatrakis, Michael; Robles, Jaime; Thota, Nitin Venkat

Date: April 29, 2026

1. Group Members and Roles

Member Name	NETID	Primary Role in Project
Diep, Ethan	ehd190001	Testing/validation support, demo logs, peer folder checks, and download-completion verification.
Giatrakis, Michael	mgg210002	Peer-side support, chunk-transfer review, retry/resume checks, and 1024-byte limit verification.
Robles, Jaime	jxr240036	Tracker-side support, tracker-file format review, command parsing checks, and configuration review.
Thota, Nitin	nvt230000	Integration and final implementation lead: tracker-peer integration, starter script, documentation, testing, and report preparation.

2. Project Overview

This project implements a TCP-based peer-to-peer file sharing system with a centralized tracker server. The tracker maintains .track files that describe shared files, file size, MD5 checksum, and the peer byte ranges currently available in the network. Peers communicate with the tracker to create tracker files, request tracker lists, fetch tracker metadata, and update their available byte ranges. After a peer receives a valid tracker file, it downloads the target file directly from other peers using multiple worker threads.

The implementation has three main parts: tracker.c for the centralized tracker server, peer.c for peer behavior, and common.c/common.h for shared socket, file, MD5, and tracker-parsing utilities. The system is designed to run on Linux or WSL Ubuntu with gcc, pthreads, and POSIX TCP sockets.

3. Code Design

3.1 Tracker Server Design

The tracker is a multi-threaded TCP server. It reads its listening port, tracker-file directory, and dead-peer timeout from sconfig. On startup, it prepares the configured tracker directory, starts listening for peer requests, and assigns each accepted connection to a detached worker thread. A single worker handles one request and closes the connection after sending the protocol response.

- LIST / REQ LIST: scans the tracker directory and replies with <REP LIST X>, one line per tracker file, and <REP LIST END>.
- GET filename.track: reads the requested tracker file, sends <REP GET BEGIN>, the tracker file contents, and <REP GET END FileMD5>.
- createtracker: creates a new .track file containing Filename, Filesize, Description, MD5, and the first full-file peer record.
- updatetracker: adds or refreshes a peer range, updates the timestamp, removes dead peers, and writes the tracker file back safely.
- Concurrency protection: tracker file reads/writes are protected with a pthread mutex so simultaneous updates do not corrupt tracker metadata.

3.2 Peer Program Design

Each peer is both a tracker client and an upload server for other peers. The peer reads clientThreadConfig.cfg for tracker port, tracker IP, and update interval. It reads serverThreadConfig.cfg for its listening port and shared folder, matching the project handout. The peer obtains its own IP automatically; an optional advertised IP line is also accepted for two-machine or WSL troubleshooting when auto-detection reports an unreachable address.

- Seed mode: a peer scans its shared folder, computes file size and MD5, and sends createtracker for every complete file it shares.
- Download mode: a peer sends GET for the .track file, verifies the tracker-data MD5, caches the tracker file, parses peer ranges, and starts downloading automatically.

- Peer selection: for each needed byte range, the peer chooses a source peer that has the range and the newest timestamp. If a TCP connection fails, it retries using refreshed tracker data.
- Chunk transfer: each segment is downloaded in 1024-byte chunks. The peer upload server rejects invalid requests or requests larger than 1024 bytes using <GET invalid>.
- Resume support: completed segments are recorded in cache .parts files. On restart, the peer can skip already completed segments and continue missing ranges.
- Completion: after all segments are downloaded, the peer verifies the final file MD5 against the tracker metadata, deletes the cached tracker/parts files, and prints a download-complete event.

4. Protocol and File Formats

Purpose	Message / Format
Create tracker	<createtracker filename filesize description md5 ip-address port-number>
Create reply	<createtracker succ>, <createtracker fail>, or <createtracker ferr>
Update tracker	<updatetracker filename start byte end byte ip-address port-number>
Update reply	<updatetracker filename succ>, <updatetracker filename fail>, or <updatetracker filename ferr>
List trackers	<REQ LIST> followed by <REP LIST X> ... <REP LIST END>
Get tracker file	<GET filename.track> followed by <REP GET BEGIN> ... <REP GET END FileMD5>
Peer chunk request	<GETCHUNK filename start byte end byte>; max response payload is 1024 bytes

Tracker files are stored in the configured tracker directory using the name filename.track. The file format contains Filename, Filesize, Description, MD5, optional comment lines beginning with #, and peer records in the form ip:port:start:end:timestamp. The parser ignores blank lines and comment lines.

5. Installation and Running Guide

5.1 Installation / Build

Required environment: Linux or WSL Ubuntu with gcc, make, pthreads, and standard POSIX socket libraries. From the project directory, run:

```
make -f makefile clean
make -f makefile
```

Expected result: gcc compiles tracker.c, peer.c, and common.c, producing the executables tracker and peer. The makefile also creates peer1 through peer13 folders with shared, cache, downloads, and logs subfolders.

5.2 Manual Run

Start the tracker: `./tracker sconfig`

Start an interactive peer: `./peer Peer1 peer1`

Start a seed peer: `./peer Peer1 peer1 --seed`

Start a downloader peer: `./peer Peer3 peer3 --download small.txt.track large.bin.track`

Interactive peer commands include LIST, GET filename.track, createtracker filename description, updatetracker filename start end, and quit. Full protocol-form createtracker/updatetracker commands are also supported for manual testing.

5.3 Automated Demo

The included starter.sh script builds the project, creates clean peer folders/configurations, starts the tracker, starts Peer1 and Peer2 as seed peers, starts Peer3 through Peer8 after 30 seconds, starts Peer9 through Peer13 after 1 minute 30 seconds, validates that Peer3 through Peer13 downloaded both files, and stops all processes. Logs are written to tracker.log and peer*/logs/run.log.

Command: `sh starter.sh`

Important expected outputs include tracker startup, tracker file creation, peer GET events, updatetracker responses, and File small.txt download complete / File large.bin download complete lines for Peer3 through Peer13.

6. Configuration Files

File	Contents
sconfig	Line 1: tracker port; Line 2: tracker directory; Line 3: dead-peer timeout. Current values: 5000, torrents, 1800.
clientThreadConfig.cfg	Line 1: tracker port; Line 2: tracker IP; Line 3: periodic update interval. Current demo value uses 900 seconds, which is 15 minutes.
serverThreadConfig.cfg	Line 1: peer listen port; Line 2: shared folder. Optional Line 3: advertised peer IP for two-machine/WSL troubleshooting only.

7. Error Handling and Reliability Features

- Socket setup and connection failures are checked for tracker and peer connections.
- connect_tcp uses bounded timeouts so a dead peer does not stall a download worker indefinitely.
- send_all and recv_all handle partial network sends/receives.
- The tracker handles duplicate tracker creation with ferr and missing tracker updates with filename ferr.
- The peer verifies tracker-file MD5 before parsing and verifies the final downloaded file MD5 before declaring completion.
- The tracker removes stale peer entries based on the configured dead-peer timeout.
- Peer downloads retry failed sources and request fresh tracker data when a listed peer is unavailable or stale.

8. User Interface and Testing Evidence

The terminal output shows necessary events such as tracker startup, tracker creation, LIST/GET requests, live progress counters during long wait/download phases, completion messages, and peer termination events. The automated demo also writes detailed logs into tracker.log and peer*/logs/run.log so the grader can confirm chunk ranges, tracker update replies, and peer behavior without cluttering the screen.

Recommended validation commands after running starter.sh:

```
grep -hE "File .*download complete|failed to download|MD5 check failed" peer*/logs/run.log
grep -hE "tracker: listening|tracker: created|tracker: GET|updatetracker" tracker.log | head -40
ls -lh peer3/shared peer9/shared peer13/shared
```

9. External Sources / Libraries

No third-party networking or file-sharing packages were used. The implementation uses standard C/POSIX libraries, including pthreads and TCP socket system calls. The MD5 implementation follows the standard RFC 1321 / RSA Data Security, Inc. reference algorithm structure.

Citation: Ronald L. Rivest, RFC 1321, The MD5 Message-Digest Algorithm, Internet Engineering Task Force, 1992, <https://www.rfc-editor.org/rfc/rfc1321>

Citation: Brian Hall, Beej's Guide to Network Programming: Using Internet Sockets, used as socket-programming reference material.